

MisterFix

Marketplace de Serviços Domésticos

RELATÓRIO TÉCNICO FINAL — SPRINT 4

App Flutter Prestador + Integração de Ponta a Ponta

Disciplina	Lab. de Desenvolvimento de Aplicações Móveis e Distribuídas
Professores	Cleiton Silva Tavares e Cristiano de Macedo Neto
Aluno	Matheus Ferreira Tilton
Sprint	Sprint 4 — Entrega Final (29/06/2026)
Repositório	Github.com/matheustilton/Mrfix

1. Visão Geral do Sistema

O MisterFix é uma plataforma de marketplace de serviços domésticos desenvolvida como projeto integrador da disciplina de Laboratório de Desenvolvimento de Aplicações Móveis e Distribuídas (LDAMD). O sistema conecta clientes que precisam de manutenção residencial a prestadores de serviço qualificados, operando sobre uma arquitetura orientada a eventos (Event-Driven Architecture — EDA).

O ecossistema é composto por três componentes principais: o backend RESTful em Node.js/Express com SQLite, um middleware orientado a mensagens (MOM) baseado em RabbitMQ, e dois aplicativos Flutter — um voltado ao cliente e outro ao prestador. A Sprint 4 encerra o ciclo de desenvolvimento, entregando o aplicativo do prestador e demonstrando o fluxo completo de ponta a ponta.

1.1 Identidade Visual dos Aplicativos

Por design, cada aplicativo adota uma paleta primária distinta para reforçar a separação de contextos:

Componente	Cor Primária	Justificativa
App Cliente (mrfix-client)	Vermelho #C82828	Urgência, ação, solicitação — o cliente dispara o evento
App Prestador (mrfix_provider)	Azul #0D4F7A	Confiança, profissionalismo — o prestador executa e conclui
Backend / MOM / Relatório	Roxo-Vinho #7B3553	Fusão das duas paletas, simbolizando a integração dos componentes

2. Arquitetura do Sistema

A arquitetura adota os princípios da Clean Architecture (MARTIN, 2019) tanto no backend quanto nos aplicativos Flutter, organizando cada componente em camadas com dependências unidirecionais. No backend, a separação em Models → Controllers → Routes garante que regras de negócio não vazem para a camada de transporte HTTP.

2.1 Componentes e Protocolos

Componente	Tecnologia	Protocolo	Responsabilidade
Backend REST	Node.js 18 + Express	HTTP/REST + JWT	Lógica de negócio, autenticação, persistência
Banco de Dados	SQLite + Sequelize ORM	Arquivo local	Persistência de usuários, pedidos, avaliações
MOM	RabbitMQ + amqpcli	AMQP 0-9-1	Comunicação assíncrona orientada a eventos
App Cliente	Flutter/Dart + Provider	HTTP + Polling	Telas do cliente: pedidos, histórico, avaliação
App Prestador	Flutter/Dart + Provider	HTTP + Polling	Telas do prestador: demandas, aceite, progresso

2.2 Camadas do App Flutter (Clean Architecture)

Ambos os aplicativos seguem a mesma organização de pacotes, com adaptações de domínio:

Camada	Pacote / Arquivos	Responsabilidade
Domain	lib/domain/entities/	Entidades puras de negócio (UserEntity, ServiceRequestEntity, ProviderSpecialtyEntity)
Data	lib/data/models/ + lib/data/datasources/	Modelos com fromJson/toJson; RemoteDataSource com chamadas HTTP
Network	lib/core/network/	ApiClient (cliente) / ApiProvider (prestador) — injeção de JWT, parse de respostas
Providers	lib/presentation/providers/	AuthProvider, ServiceRequestProvider, ThemeProvider (ChangeNotifier)
Screens	lib/presentation/screens/	Telas Flutter: splash, login, register, home, requests, detail, profile
Widgets	lib/presentation/widgets/	Componentes reutilizáveis: AppTextField, PrimaryButton, StatusBadge, TappableCard
Constants	lib/core/constants/	AppConstants (cores, status, ícones), AppTheme (light/dark)

3. Aplicativo Flutter — Cliente

O aplicativo do cliente (mrfix-client) foi desenvolvido para Android e iOS via Flutter 3. Ele permite ao usuário solicitar serviços domésticos, acompanhar o status dos pedidos em tempo real e avaliar o prestador ao final. Todas as telas consomem a API REST do backend via HTTPS e atualizam o estado local através do padrão Provider.

3.1 Telas Implementadas

Tela	Funcionalidade
SplashScreen	Animação de entrada com verificação automática de token JWT — redireciona para Login ou Home conforme autenticação
LoginScreen	Autenticação com e-mail e senha; tabs "Sou Cliente" / "Sou Prestador" (visuais); toggle dark mode; link para cadastro
RegisterScreen	Cadastro com nome, e-mail, senha e seleção de gênero (RadioListTile); envia role: client
HomeScreen	Dashboard com categorias horizontais, toggle "apenas prestadoras mulheres" e lista de pedidos ativos com polling a cada 5 s
RequestsScreen	TabBar com 3 abas: Ativas / Concluídas / Canceladas — lista completa de pedidos com StatusBadge
NewRequestScreen	Formulário de criação: categoria (chip selector), descrição, endereço, agendamento, preferência de gênero; slide-up animation
RequestDetailScreen	Status card, progress steps (4 etapas), info card, provider card, cancelamento e avaliação com estrelas (1–5)

3.2 Polling Assíncrono de Estado

Como mecanismo de atualização assíncrona, o app do cliente implementa polling periódico usando `Timer.periodic` com intervalo de 5 segundos (`AppConstants.pollingInterval`). Ao entrar na `HomeScreen`, `startPolling()` é chamado; ao sair, `stopPolling()` encerra o timer para economizar recursos.

O método `_silentRefresh()` busca `GET /service-requests` sem exibir indicador de carregamento, atualizando silenciosamente a lista e — caso haja um pedido selecionado aberto em `RequestDetailScreen` — substitui o objeto local pelo retorno atualizado. Essa abordagem é compatível com a exigência da Sprint 3 de "atualização assíncrona de estado sem exigir ação manual do usuário".

Nota de evolução: o polling será substituído por WebSocket (Socket.io) ou Firebase Cloud Messaging na Sprint futura, eliminando requisições desnecessárias quando não há mudança de estado.

4. Aplicativo Flutter — Prestador

O aplicativo do prestador (`mrfix_provider`) foi desenvolvido como aplicação independente para reforçar a separação de contextos e identidade visual. Ele permite ao profissional gerenciar demandas disponíveis, aceitar ou rejeitar pedidos, atualizar o progresso do serviço e visualizar seu histórico e perfil.

4.1 Telas Implementadas

Tela	Funcionalidade
SplashScreen	Animação com logo azul, verificação de JWT e redirecionamento para Login ou HomeScreen do prestador
LoginScreen	Autenticação do prestador; formulário idêntico ao do cliente porém com paleta azul
RegisterScreen	Cadastro com role: provider (corrigido na Sprint 4); campo de gênero via <code>RadioListTile</code>
HomeScreen	Header azul com avatar, nome e avaliação; <code>TabBar</code> com 3 abas: Disponíveis / Meus Pedidos / Especialidades; FAB para nova especialidade
RequestDetailScreen	Detalhes da solicitação com botões contextuais: Aceitar / Iniciar Serviço / Concluir Serviço / Cancelar conforme status
RequestsScreen	Lista filtrável de pedidos em andamento e histórico do prestador com <code>StatusBadge</code> e preço acordado
AddSpecialtyScreen	Formulário para adicionar especialidade: categoria (<code>DropDownButton</code> filtrando já cadastradas), preço médio, anos de experiência e bio
ProfileScreen	Exibição do perfil do prestador: avaliação média, contagem de avaliações, lista de especialidades cadastradas e ações de edição

4.2 Lógica de Separação de Pedidos

O backend retorna, para um prestador autenticado em `GET /service-requests`, tanto os pedidos onde ele já está vinculado (`provider_id` = seu ID) quanto todos os pendentes compatíveis com seu gênero (`status` = `pending`, `preferred_gender` IN [`any`, seu_gênero]). O `RemoteDataSource` do app prestador divide esse conjunto em dois:

- `getAvailableRequests()`: retorna apenas os itens com `status` == "pending" e `provider` == null — exibidos na aba "Disponíveis"
- `getMyRequests()`: retorna todos os demais (`provider` != null ou `status` != "pending") — exibidos em "Meus Pedidos"

Essa separação client-side elimina a necessidade de rotas extras no backend e garante que o prestador não veja seus próprios pedidos como disponíveis para aceitar novamente.

4.3 Polling Assíncrono do Prestador

O polling do app prestador funciona de forma análoga ao do cliente: `_silentRefresh()` chama `getAvailableRequests()` e `getMyRequests()` em paralelo, compara os IDs retornados com a lista anterior para calcular `newRequestCount` (badge de notificação) e atualiza o estado sem bloqueio de UI.

5. Backend REST e Middleware de Mensagens

5.1 Endpoints REST Implementados

Método	Rota	Auth	Descrição
POST	/api/auth/register	Pública	Cadastro de usuário (client ou provider)
POST	/api/auth/login	Pública	Autenticação com retorno de JWT
GET	/api/auth/me	JWT	Dados do usuário autenticado
GET	/api/categories	Pública	Listagem de categorias de serviço
GET	/api/providers	Pública	Listagem de prestadores com filtros de gênero e categoria
GET	/api/providers/:id	Pública	Perfil de um prestador específico
POST	/api/providers/specialties	JWT (provider)	Cadastrar especialidade do prestador
GET	/api/providers/me/specialties	JWT (provider)	Especialidades do prestador autenticado
POST	/api/service-requests	JWT (client)	Criar nova solicitação de serviço
GET	/api/service-requests	JWT	Listar pedidos (filtrado por role: cliente vê os seus; prestador vê os seus + pendentes)
GET	/api/service-requests/:id	JWT	Detalhe de uma solicitação (com autorização por role)
PATCH	/api/service-requests/:id/status	JWT	Atualizar status (accepted / in_progress / completed / cancelled)
POST	/api/ratings	JWT (client)	Submeter avaliação ao prestador
GET	/api/ratings/user/:userId	Pública	Avaliações de um prestador
GET	/health	Pública	Health check da API

5.2 Eventos do RabbitMQ (MOM)

O sistema publica eventos em quatro filas durável (`durable: true`, `persistent: true`) via `amqp-lib`. Cada evento transporta um payload JSON com timestamp automático. Os consumidores processam as mensagens com `ack/nack` para garantia de entrega.

Fila / Evento	Produtor	Consumidor	Payload Principal
service_request.created	Backend (POST /service-requests)	requestCreatedConsumer	requestId, clientId, categoryId, categoryName, title, preferredGender

service_request.accepted	Backend (PATCH /status → accepted)	requestAcceptedConsumer	requestId, clientId, providerId, providerName
service_request.status_changed	Backend (PATCH /status — qualquer transição)	statusChangedConsumer	requestId, clientId, providerId, oldStatus, newStatus, actorRole
service_request.completed	Backend (PATCH /status → completed)	(statusChangedConsumer)	requestId, clientId, providerId, completedAt

5.3 Comportamento dos Consumidores

O requestCreatedConsumer consulta o banco para encontrar prestadores elegíveis (role = provider, is_active = true, especialidade na categoria solicitada, gênero compatível com preferred_gender) e registra em log quais profissionais seriam notificados. Em produção, esse passo dispararia push notifications via Firebase Cloud Messaging.

O statusChangedConsumer determina o destinatário da notificação pelo princípio de notificação cruzada: se quem agiu foi o prestador (actorRole = provider), notifica o cliente, e vice-versa. Mensagens legíveis são geradas por status: "Sua solicitação foi aceita!", "Serviço em andamento", "Serviço concluído!".

Todos os consumidores usam ch.ack(msg) em caso de sucesso e ch.nack(msg, false, true) para reprocessamento em caso de erro transitório, garantindo pelo menos uma entrega (at-least-once delivery).

6. Fluxo Completo de Ponta a Ponta

O fluxo integrado demonstra como os três componentes cooperam de forma assíncrona, desde a criação do pedido pelo cliente até a conclusão pelo prestador:

Passo	Ator	Ação / Evento
1	Cliente (App)	Preenche NewRequestScreen: seleciona categoria, descreve problema, informa endereço e preferência de gênero
2	App Cliente → Backend	POST /api/service-requests → backend cria registro com status=pending
3	Backend → RabbitMQ	publishRequestCreated() envia evento service_request.created para a fila
4	RabbitMQ → Consumer	requestCreatedConsumer identifica prestadores elegíveis e registra log (produção: push notification)
5	App Prestador (polling)	A cada 5 s, getAvailableRequests() detecta novo pedido; newRequestCount incrementa; aba "Disponíveis" atualiza
6	Prestador (App)	Abre RequestDetailScreen → toca em "Aceitar Pedido"
7	App Prestador → Backend	PATCH /api/service-requests/:id/status { status: "accepted" }
8	Backend → RabbitMQ	publishRequestAccepted() + publishStatusChanged() publicam dois eventos simultâneos
9	Consumer	requestAcceptedConsumer notifica o cliente; statusChangedConsumer registra a transição pending → accepted
10	App Cliente (polling)	Polling detecta status=accepted no próximo ciclo; RequestDetailScreen exibe ProgressStep "Aceito" preenchido e card do prestador

11	Prestador (App)	"Iniciar Serviço" → PATCH status=in_progress → evento published → cliente vê "Em andamento"
12	Prestador (App)	"Concluir Serviço" → PATCH status=completed → publishRequestCompleted() publicado
13	App Cliente	Polling detecta completed; botão "Avaliar prestador" aparece em RequestDetailScreen
14	Cliente (App)	Modal de avaliação com estrelas 1–5 e comentário → POST /api/ratings

O fluxo completo (passos 1–14) é demonstrado no screencast de 4 minutos entregue junto a este relatório, com os dois emuladores rodando simultaneamente.

7. Decisões de Design

7.1 Dois Aplicativos Independentes

A decisão de manter dois projetos Flutter separados, ao invés de um único app com perfis, segue a recomendação de RICHARDSON (2018) de separação de contextos (Bounded Contexts). Cada aplicativo tem seu próprio AppConstants, ApiClient, tema e fluxo de navegação. Isso elimina condicionais de role em toda a árvore de widgets e permite que as identidades visuais evoluam independentemente.

7.2 Polling vs. WebSocket vs. MOM Push

Optou-se por polling HTTP no lado dos apps Flutter por três razões pragmáticas: compatibilidade imediata com a API REST já implementada, ausência de necessidade de infraestrutura adicional (WebSocket server), e intervalo de 5 segundos suficientemente curto para a experiência de uso esperada. Conforme HOHPE e WOOLF (2003), polling é um padrão legítimo de integração quando a latência tolerada é maior que o overhead de requisição. O MOM (RabbitMQ) opera no lado do servidor para descoberta de prestadores e logging de eventos, preparando a arquitetura para futuras integrações push.

7.3 Work Queue com Persistência no RabbitMQ

A configuração durable: true nas filas e persistent: true nas mensagens garante que eventos não sejam perdidos em caso de reinicialização do broker. Essa escolha segue o padrão Durable Queue descrito por HOHPE e WOOLF (2003), adequado a sistemas onde a perda de um evento de negócio (pedido criado, aceite realizado) tem impacto direto na experiência do usuário.

7.4 Degradação Graciosa do MOM

O backend inicializa mesmo que o RabbitMQ esteja indisponível, exibindo aviso de warning e continuando a operar como API REST pura. O método publish() captura erros de conexão e os registra sem lançar exceção para a camada HTTP. Essa resiliência é essencial em ambientes de desenvolvimento onde o container do RabbitMQ pode não estar ativo.

7.5 Filtro de Gênero como Feature de Segurança

O campo preferred_gender nas solicitações implementa um filtro de segurança que permite ao cliente solicitar atendimento exclusivamente por prestadoras mulheres. No backend, o listRequests filtra pedidos visíveis ao prestador por preferred_gender IN [any, gênero_do_prestador], garantindo que apenas profissionais elegíveis vejam a demanda. O app do cliente exibe esse filtro com ícone de escudo (Icons.shield_rounded) para comunicar claramente o propósito de segurança.

8. Dificuldades Encontradas e Soluções Adotadas

Dificuldade	Solução Adotada
Role incorreto no cadastro do prestador: register() enviava role: "client" no app do prestador, criando contas sem acesso a pedidos disponíveis.	Correção cirúrgica no RemoteDataSource do mrfix_provider: role alterado para "provider". Bug identificado comparando o comportamento do backend (role = client → não vê pedidos disponíveis) com o esperado.
Rota /service-requests/available inexistente no backend: o app do prestador chamava essa rota que retornava 404, impossibilitando a listagem de demandas.	Refatoração para usar GET /service-requests (rota existente) e separar os conjuntos client-side: available = {provider == null && status != "pending"}, myRequests = os demais.
Polling duplicando pedidos nas abas "Disponíveis" e "Meus Pedidos": o mesmo pedido aceito aparecia em ambas.	getMyRequests() passou a filtrar apenas pedidos onde provider != null ou status != "pending", eliminando a interseção com getAvailableRequests().
Nome do app inconsistente: splash_screen e requests_screen do cliente exibiam "Marido de Aluguel" em vez de "MisterFix".	Substituição das strings hardcoded pelas referências corretas. Identificado por revisão textual do código-fonte.
Constante border ausente no AppConstants do cliente: o arquivo do prestador definia border como alias de borderLight, mas o cliente não tinha o alias.	Adicionado static const Color border = Color(0xFFE8E8E8) ao AppConstants do cliente para paridade de API entre os dois apps.
RabbitMQ indisponível durante desenvolvimento derrubava o servidor Express.	Bloco try/catch no start() isola a inicialização do MOM; o servidor sobe normalmente com warning, garantindo que a API REST funcione independentemente.

9. Reflexão sobre os Padrões Estudados

9.1 Event-Driven Architecture (EDA)

A adoção de EDA no MisterFix tornou os componentes fracamente acoplados: o backend não precisa saber quantos consumidores escutam `service_request.created`, e novos consumidores (ex.: serviço de e-mail, analytics) podem ser adicionados sem modificar o produtor. Conforme COULOURIS et al. (2011), essa propriedade é essencial em sistemas distribuídos onde os componentes devem evoluir de forma independente.

Na prática, o padrão EDA permitiu que o fluxo de notificação ao prestador fosse implementado no consumer sem alterar o controlador `createRequest` — demonstração concreta do princípio Open/Closed (MARTIN, 2019).

9.2 MOM com RabbitMQ

O RabbitMQ opera no padrão Work Queue (Point-to-Point), onde cada mensagem é entregue a exatamente um consumidor. Para o domínio do MisterFix, isso é adequado: cada evento de "pedido criado" deve ser processado uma única vez pelo serviço de matching de prestadores. Se o padrão fosse Pub/Sub (Fanout Exchange), múltiplos consumidores processariam o mesmo evento — útil para futuros serviços de analytics ou auditoria.

9.3 Clean Architecture nos Apps Flutter

A separação em Domain → Data → Presentation garantiu que mudanças na API REST (ex.: renomeação de campos JSON) fossem absorvidas exclusivamente na camada de modelos (fromJson), sem impactar telas ou providers. Conforme BAILEY (2023), essa abordagem é especialmente valiosa em apps Flutter onde a UI é reconstruída frequentemente via hot reload.

Os Providers (ChangeNotifier) atuam como a camada de estado, desacoplando a lógica de negócio dos widgets. Ao invés de StatefulWidget com setState() por toda a árvore, o estado é centralizado e propagado via Consumer<T>, facilitando testes unitários futuros.

10. Pendências e Próximos Passos

O sistema entregueado na Sprint 4 cobre todos os critérios obrigatórios do projeto. As pendências abaixo representam evoluções naturais identificadas ao longo do desenvolvimento:

Pendência	Impacto	Solução Proposta
Push notifications reais ao prestador	Alto — prestador depende de polling para ver novos pedidos	Firebase Cloud Messaging (FCM) integrado ao consumer requestCreatedConsumer
Substituição do polling por WebSocket	Médio — reduz tráfego desnecessário e latência de atualização	Socket.io no backend + web_socket_channel no Flutter
Telas de Chat cliente ↔ prestador	Médio — comunicação direta entre as partes sem telefone	Novo modelo Message + fila RabbitMQ + tela ChatScreen
Pagamento integrado	Alto — monetização da plataforma	Stripe ou Pagar.me com webhook de confirmação
Deploy em nuvem	Médio — acesso fora do localhost	Railway (backend) + CloudAMQP (RabbitMQ gerenciado) + variáveis de ambiente
Testes automatizados	Alto — confiabilidade em produção	Jest (backend) + flutter_test + mockito (apps)

11. Instruções de Execução

11.1 Backend

```
cd MrFix/backend
npm install
# Subir RabbitMQ via Docker (opcional):
docker run -d --name rabbitmq -p 5672:5672 -p 15672:15672 rabbitmq:3-management
node src/server.js
```

11.2 App Cliente

```
cd MrFix/frontend/mrfix-client
flutter pub get
# Ajustar baseUrl em lib/core/constants/app_constants.dart se necessário
flutter run
```

11.3 App Prestador

```
cd MrFix/frontend/mrfix-provider/mrfix_provider
flutter pub get
flutter run
```

Repositório Git: github.com/matheustitton/mrfix | Sprint 4 — 03/07/2026